



**UNIVERSIDAD
CATÓLICA DE
TEMUCO**

DEPARTAMENTO DE
INGENIERÍA INFORMÁTICA
FACULTAD DE INGENIERÍA

Evaluación experimental de eficiencia energética y rendimiento en clústeres de Alta Disponibilidad con K3s y MicroK8s

por

José Ignacio Delpino Muñoz

Trabajo presentado a la
Facultad de Ingeniería de la Universidad Católica de Temuco
Para optar al título de Ingeniero Civil Informático.

- Temuco, Septiembre 2026 -

Índice general

1. Introducción	1
1.1. Problema	1
1.2. Estado del Arte	4
1.3. Objetivos	7
1.3.1. Objetivo General	7
1.3.2. Criterios SMART del objetivo general	7
1.3.3. Objetivos Específicos	8
1.3.4. Matriz de trazabilidad	9
1.4. Análisis de Requerimientos	10
1.4.1. Requerimientos funcionales de arquitectura y orquestación	10
1.4.2. Requerimientos de automatización e infraestructura	10
1.4.3. Requerimientos de observabilidad energética	11
1.4.4. Requerimientos de observabilidad de recursos y rendimiento	11
1.4.5. Requerimientos de evaluación experimental	12
1.4.6. Requerimientos de resiliencia	14
1.4.7. Requerimientos de contraste con nube pública	14
1.4.8. Evaluación crítica de alternativas	14
A. Tablas	18
A.1. Matriz de objetivos específicos y requisitos definidos	18
A.2. Métricas e instrumentos de medición	19
B. Enlace a backlog	20

Capítulo 1

Introducción

1.1. Problema

La adopción de arquitecturas basadas en microservicios y contenedores ha incrementado la necesidad de administrar aplicaciones distribuidas mediante mecanismos de automatización, configuración declarativa, escalamiento y recuperación ante fallos. La literatura identifica la orquestación, el rendimiento y la monitorización como desafíos relevantes de los entornos contenerizados, especialmente cuando las aplicaciones se ejecutan sobre infraestructuras distribuidas o con recursos limitados (Casalicchio & Iannucci, 2020). En este contexto, Kubernetes se ha consolidado como una plataforma de código abierto, portable y extensible para administrar cargas de trabajo y servicios contenerizados, proporcionando automatización y configuración declarativa (Kubernetes Authors, 2024).

La relevancia actual de Kubernetes también se refleja en su nivel de adopción. La Cloud Native Computing Foundation (CNCF), en su encuesta anual correspondiente a 2025, informa que el 82 % de los usuarios de contenedores encuestados ejecuta Kubernetes en producción. El mismo informe señala que el 66 % de las organizaciones que alojan modelos de inteligencia artificial generativa utiliza Kubernetes para gestionar parte o la totalidad de sus cargas de inferencia (Cloud Native Computing Foundation, 2026). Esta expansión hace que las decisiones relacionadas con el uso eficiente de recursos de cómputo adquieran importancia tanto en centros de datos propios como en servicios de computación en la nube.

Desde una perspectiva histórica, Kubernetes recoge aprendizajes derivados de sistemas de administración de contenedores desarrollados previamente por Google, entre ellos, Burns et al. (2016) describen esta evolución y destacan la importancia de abstraer la gestión de recursos y servicios distribuidos para simplificar la operación a gran escala. De este modo, la evolución de la orquestación no solo ha buscado automatizar el despliegue, sino también facilitar la administración de sistemas distribuidos cada vez más complejos.

Sin embargo, la adopción de plataformas de orquestación introduce un problema relevante de eficiencia de recursos. El consumo de CPU, memoria, almacenamiento y red asociado al plano de control, a los servicios auxiliares y a los mecanismos de alta disponibilidad forma parte del costo ope-

rativo de una infraestructura, especialmente cuando se utilizan equipos con recursos limitados. En particular, la eficiencia energética se ha transformado en un aspecto de interés debido a la necesidad de relacionar el trabajo computacional realizado con la energía requerida para realizarlo (Pijnacker et al., 2025). Este interés también se refleja en propuestas orientadas a la programación de cargas de trabajo sensible a criterios de sostenibilidad energética en entornos Kubernetes distribuidos (Huang et al., 2026). Por lo tanto, una comparación basada exclusivamente en utilización de CPU o memoria no permite determinar por sí sola qué plataforma presenta una mejor eficiencia energética.

En escenarios de recursos limitados, las distribuciones ligeras de Kubernetes constituyen una alternativa para reducir la sobrecarga de la plataforma. Koziolk y Eskandani (2023) compararon MicroK8s, K3s, k0s y MicroShift considerando consumo mínimo de recursos y rendimiento del plano de control y del plano de datos bajo escenarios de carga. Sus resultados muestran que las distribuciones presentan diferencias medibles en utilización de recursos y rendimiento, lo que evidencia que la elección de una distribución puede modificar el comportamiento de una infraestructura bajo determinadas cargas de trabajo.

Investigaciones posteriores también han estudiado las diferencias entre distribuciones ligeras desde la perspectiva de eficiencia de recursos y desempeño. Yakubov y Hästbacka (2025) realizaron una evaluación empírica de distribuciones orientadas a Edge Computing, considerando variables como CPU, memoria, uso de disco, *throughput* y latencia. Estos antecedentes respaldan la necesidad de utilizar múltiples métricas de evaluación en lugar de concluir a partir del consumo de un único recurso.

En particular, K3s y MicroK8s presentan mecanismos integrados para construir clústeres de Alta Disponibilidad. K3s permite formar un clúster HA utilizando tres o más nodos *server* con *etcd* embebido, manteniendo un número de nodos que permita conservar el quórum del almacén de estado distribuido (K3s Project Authors, 2026). MicroK8s, por su parte, habilita automáticamente la Alta Disponibilidad en clústeres de tres o más nodos y utiliza Dqlite como almacén de estado distribuido (Canonical, 2026a). Estas diferencias arquitectónicas son relevantes para el presente estudio, debido a que los mecanismos de replicación y consenso forman parte de la infraestructura que será analizada durante los experimentos.

A la necesidad de mantener el estado distribuido se suma la necesidad de conservar un punto de acceso estable a la API del clúster. En una infraestructura local, una solución posible consiste en proporcionar una dirección IP virtual mediante un protocolo de redundancia de routers. VRRP define un mecanismo de elección que permite transferir dinámicamente la responsabilidad asociada a una dirección virtual desde un nodo maestro hacia un nodo de respaldo cuando el primero deja de estar disponible (Nadas, 2010). En este proyecto, dicha capacidad se utilizará mediante Keepalived para representar un punto de acceso estable al plano de control.

Un segundo desafío corresponde a la observabilidad energética. La medición del consumo energético en entornos de Kubernetes requiere definir explícitamente el nivel de observación y la relación entre la energía registrada y el trabajo realizado. En este contexto, Kepler constituye una alternativa de observabilidad energética integrada con Kubernetes, capaz de proporcionar métricas a distintos niveles, incluyendo nodo, contenedor, proceso, máquina virtual y pod (Sustainable Computing IO, 2026). Sin embargo, estudios experimentales han identificado limitaciones en la precisión de determi-

nadas métricas cuando se utilizan para estimar o atribuir energía a nivel de contenedor. En particular, Pijnacker et al. (2025) reportaron una precisión no satisfactoria de las métricas evaluadas a nivel de contenedor y propusieron KubeWatt como una alternativa para determinados escenarios.

Para el presente proyecto, la medición energética se realizará directamente sobre el **host físico** que ejecutará las máquinas virtuales mediante QEMU/KVM. Para ello, se utilizará Scaphandre como herramienta de recolección de energía y potencia, empleando los contadores energéticos disponibles mediante el subsistema *powercap*/RAPL (The Linux Kernel Documentation, 2022; Hubblo, 2026a, 2026b). Esta decisión permite trabajar con una medición energética a nivel de host, manteniendo separada la energía contabilizada por la plataforma física de las métricas de utilización observadas dentro de las máquinas virtuales y de los clústeres Kubernetes.

A partir de lo anterior, el problema de investigación se define como la **falta de evidencia experimental comparable y reproducible sobre la relación entre energía contabilizada, rendimiento y resiliencia en clústeres de Alta Disponibilidad implementados con K3s y MicroK8s bajo condiciones de carga controladas**. Esta falta de evidencia dificulta establecer cómo se comportan ambas distribuciones cuando se consideran simultáneamente la energía requerida durante la ejecución, el trabajo computacional realizado y la capacidad de recuperación frente a fallos controlados.

La investigación considera además una segunda dimensión asociada al tipo de infraestructura. En el modelo IaaS, el consumidor dispone de recursos virtualizados de procesamiento, almacenamiento y red, pero no administra directamente la infraestructura física subyacente (Mell & Grance, 2011). Por esta razón, **no se plantea una comparación experimental directa entre una infraestructura local y una nube pública**. La experimentación principal se realizará sobre una infraestructura *On-Premise* controlada, mientras que los resultados podrán ser contrastados posteriormente con evidencia secundaria publicada para entornos de nube pública. Este contraste tendrá carácter contextual y no será utilizado para inferir directamente el consumo eléctrico físico de una infraestructura administrada por un proveedor cloud.

Percepción de las partes interesadas

Las necesidades de las partes interesadas se identifican, para esta primera etapa, a partir de las problemáticas descritas en la literatura especializada y no mediante entrevistas directas. Esta distinción permite diferenciar la evidencia secundaria de una percepción obtenida directamente de actores del proyecto.

Desde la perspectiva de administradores de infraestructura y profesionales DevOps, resulta relevante disponer de configuraciones de clúster que puedan ser desplegadas de forma reproducible y que mantengan Alta Disponibilidad sin requerir una infraestructura excesivamente compleja. Esta necesidad se relaciona con el uso de distribuciones ligeras y automatización de infraestructura.

Desde la perspectiva de profesionales vinculados con sostenibilidad y gestión de costos, existe interés en disponer de métricas que permitan relacionar consumo energético con utilización de recursos y carga de trabajo. La literatura sobre observabilidad energética en Kubernetes evidencia la importancia de disponer de una metodología de medición apropiada para fundamentar decisiones de optimización (Pijnacker et al., 2025).

Finalmente, desde la perspectiva de investigadores y profesionales SRE, es relevante conocer los compromisos existentes entre rendimiento, resiliencia y eficiencia de recursos. Los estudios comparativos de distribuciones ligeras muestran que una menor huella de recursos no implica necesariamente el mejor rendimiento en todas las cargas de trabajo (Koziolk & Eskandani, 2023; Yakubov & Hästbacka, 2025).

Pregunta de investigación

¿Cómo difieren K3s y MicroK8s, configurados en Alta Disponibilidad, en la energía contabilizada por el host y en su rendimiento bajo condiciones de carga controladas, y qué compromisos entre eficiencia y resiliencia pueden identificarse a partir de sus resultados?

1.2. Estado del Arte

El estado del arte se estructura en cuatro dimensiones relacionadas con el problema a investigar: distribuciones ligeras de Kubernetes, Alta Disponibilidad, observabilidad energética e infraestructura de nube pública.

Distribuciones ligeras y rendimiento

(Koziolk & Eskandani, 2023) realizaron una comparación experimental de MicroK8s, K3s, k0s y MicroShift, considerando su utilización mínima de recursos y el desempeño del plano de control y del plano de datos. El estudio fue desarrollado sobre equipos Intel NUC y contempló escenarios de estrés. Entre sus resultados, los autores reportan diferencias relevantes entre las distribuciones en utilización de CPU, memoria y rendimiento. K3s destaca por presentar una baja huella de recursos en determinadas configuraciones. Sin embargo, el estudio se concentra en la comparación de distribuciones y no analiza como eje principal el costo energético de mantener mecanismos de Alta Disponibilidad y un punto de acceso redundante al plano de control.

(Yakubov & Hästbacka, 2025) amplían este tipo de evaluación hacia escenarios de Edge Computing, comparando distintas alternativas mediante métricas de CPU, memoria, disco, *throughput* y latencia bajo diferentes cargas. Sus resultados muestran que el comportamiento relativo depende de la métrica evaluada y del escenario de carga. Este antecedente es relevante para el presente proyecto porque sustenta la necesidad de utilizar múltiples métricas de desempeño en lugar de concluir a partir del consumo de un único recurso.

Comparaciones de carácter más aplicado, como la publicada por el SFEIR Institute, también contrastan K3s y MicroK8s frente a Kubernetes estándar desde una perspectiva de adopción práctica, lo que refuerza que la elección entre distribuciones ligeras es un tema de interés tanto académico como de la industria (SFEIR Institute, 2025).

A partir de estos trabajos se identifica una primera brecha: existe evidencia comparativa sobre utilización de recursos y rendimiento, pero el presente proyecto busca estudiar específicamente K3s y

MicroK8s bajo una configuración de Alta Disponibilidad controlada y utilizando el consumo energético como variable central de eficiencia.

Alta Disponibilidad en K3s y MicroK8s

La arquitectura HA de K3s con *etcd* embebido utiliza tres o más nodos *server*. La documentación oficial señala que los clústeres HA con *etcd* embebido requieren tres o más nodos servidores y que un número impar favorece el mantenimiento del quórum (K3s Project Authors, 2026). Esta configuración permite estudiar directamente el costo operacional de la replicación del estado dentro de los mismos nodos que proporcionan el plano de control.

MicroK8s sigue un enfoque diferente: en clústeres de tres o más nodos la Alta Disponibilidad se habilita automáticamente y Dqlite se utiliza como almacén de estado. La documentación de Canonical indica que los nodos del clúster participan en la replicación del estado y que la disponibilidad depende del mantenimiento del quórum (Canonical, 2026a). Además, la documentación de recuperación del clúster especifica procedimientos asociados a la pérdida de quórum (Canonical, 2026b).

La diferencia arquitectónica entre *etcd* embebido en K3s y Dqlite en MicroK8s constituye, por tanto, una dimensión concreta de comparación. La contribución propuesta no consiste en demostrar que uno de estos mecanismos es universalmente superior, sino en observar cómo sus respectivas implementaciones se comportan bajo condiciones experimentales equivalentes.

Observabilidad y medición energética

La observabilidad constituye un componente fundamental del diseño experimental, debido a que las variables energéticas, de utilización de recursos y de rendimiento representan diferentes dimensiones del comportamiento de los clústeres. En consecuencia, no resulta conveniente utilizar una única herramienta para obtener todas las métricas, sino establecer una separación explícita entre las fuentes de medición y el nivel de observación correspondiente.

En el ámbito de la energía, Pijnacker et al. (2025) estudiaron la observabilidad energética de cargas de trabajo en Kubernetes y analizaron la precisión de métricas obtenidas mediante Kepler. Sus resultados muestran que la utilización de métricas de atribución energética a nivel de contenedor requiere considerar cuidadosamente el método de estimación y el nivel de observación. La documentación de Kepler confirma que la herramienta proporciona métricas relacionadas con energía y potencia en distintos niveles de abstracción (Sustainable Computing IO, 2026). Estos antecedentes resultan relevantes para delimitar el alcance de la medición del presente proyecto.

Para este estudio, la energía contabilizada se medirá a nivel del host físico mediante los contadores disponibles en la plataforma y utilizando Scaphandre como herramienta de recolección (The Linux Kernel Documentation, 2022; Hubblo, 2026a, 2026b). La medición energética constituirá una dimensión independiente de las métricas de utilización de recursos y del rendimiento de las aplicaciones.

Las métricas de utilización del sistema serán obtenidas mediante *Node Exporter*, herramienta del ecosistema Prometheus orientada a exponer información del sistema operativo y del hardware, incluyendo métricas de CPU, memoria, sistema de archivos y red (Prometheus Authors, 2026b). Estas

métricas permitirán caracterizar el comportamiento de los nodos durante los distintos escenarios de carga y relacionarlo posteriormente con la energía contabilizada.

Las series temporales de estas mediciones serán almacenadas mediante Prometheus, cuya arquitectura utiliza datos temporales identificados por métricas y etiquetas, permitiendo consultar y comparar la evolución de las variables durante cada ejecución (Prometheus Authors, 2026a). De esta forma, será posible alinear temporalmente las mediciones energéticas y de recursos con los periodos en que se ejecuten las pruebas de carga.

El rendimiento de la aplicación será evaluado mediante una herramienta de pruebas de carga, utilizando *k6* como alternativa de referencia. La herramienta proporciona métricas incorporadas para solicitudes HTTP, incluyendo número de solicitudes, duración de las solicitudes y tasa de errores, y permite conservar resultados agregados o datos con marcas temporales para su análisis posterior (Grafana Labs, 2026a, 2026b). Estas métricas permitirán caracterizar el *throughput*, la latencia y la tasa de error de la carga de trabajo utilizada en ambos clústeres.

Finalmente, la evaluación de resiliencia se apoyará en los mecanismos de estado y eventos de Kubernetes, complementados con registros temporales de las pruebas de fallo. Esto permitirá determinar el instante de indisponibilidad de un nodo, el momento de detección de la falla, la recuperación del servicio y el impacto sobre las solicitudes procesadas.

En conjunto, esta separación permite establecer una metodología de observabilidad donde cada herramienta cumple una función específica: Scaphandre para energía y potencia del host, Node Exporter para utilización de recursos, Prometheus para almacenamiento y consulta de series temporales, y *k6* para la generación y caracterización de la carga de trabajo. Esta arquitectura evita atribuir a una única herramienta responsabilidades que corresponden a distintos niveles de observación.

Infraestructura local y nube pública

Según NIST, el modelo IaaS permite al consumidor aprovisionar recursos fundamentales de procesamiento, almacenamiento y redes, manteniendo control sobre el sistema operativo y las aplicaciones, pero sin administrar directamente la infraestructura física subyacente (Mell & Grance, 2011). Esta característica es particularmente relevante para el análisis energético: en un entorno *On-Premise*, el investigador puede controlar las condiciones del hardware experimental, en cambio, en una nube pública el proveedor abstrae la infraestructura física.

En consecuencia, el presente proyecto utilizará la nube pública como **contexto comparativo secundario**. La evidencia experimental será obtenida en la infraestructura local, mientras que los antecedentes de nube pública serán utilizados para discutir la transferibilidad y las limitaciones de los resultados. Esta decisión metodológica evita presentar como equivalentes mediciones obtenidas mediante instrumentos y niveles de observación diferentes.

Brechas identificadas

A partir del análisis realizado, se identifican las siguientes brechas que fundamentan la propuesta:

1. Existen estudios que comparan distribuciones ligeras de Kubernetes en términos de consumo de recursos y rendimiento, pero existe una oportunidad de estudiar específicamente K3s y MicroK8s bajo una configuración de Alta Disponibilidad controlada (Koziolek & Eskandani, 2023; SFEIR Institute, 2025; Yakubov & Hästbacka, 2025).
2. La arquitectura HA introduce mecanismos adicionales de replicación, consenso y recuperación cuyo comportamiento energético debe formar parte del escenario experimental, en lugar de analizar únicamente el costo de ejecutar una distribución en modo individual (Canonical, 2026a; K3s Project Authors, 2026).
3. La observabilidad energética a nivel de contenedor presenta limitaciones documentadas en la literatura, por lo que resulta necesario definir explícitamente el nivel de medición de acuerdo con el objetivo experimental. En este proyecto se prioriza la medición energética a nivel de host, evitando atribuir directamente la energía contabilizada por la plataforma física a contenedores o máquinas virtuales individuales (Pijnacker et al., 2025).
4. La evidencia experimental local y la evidencia reportada para nube pública deben mantenerse conceptualmente separadas, debido a que en IaaS el hardware físico subyacente no se encuentra directamente bajo control del consumidor (Mell & Grance, 2011).

Estas brechas permiten formular un estudio centrado en la relación entre energía, rendimiento y resiliencia, evitando convertir la investigación en una simple comparación funcional entre distribuciones.

1.3. Objetivos

1.3.1. Objetivo General

Evaluar experimentalmente la energía contabilizada a nivel del host y el rendimiento operacional de clústeres de Alta Disponibilidad implementados con K3s y MicroK8s, bajo condiciones de carga controladas y sobre una infraestructura homogénea y reproducible, para identificar los compromisos entre eficiencia energética, desempeño y resiliencia.

Como complemento del análisis experimental, los resultados obtenidos en la infraestructura local podrán ser contrastados con evidencia secundaria disponible sobre implementaciones de infraestructura como servicio (*IaaS*) en nube pública. Este contraste se considerará contextual y no constituirá una segunda medición experimental del consumo energético, debido a que no se dispone del mismo nivel de acceso a la infraestructura física subyacente de un proveedor de nube (Mell & Grance, 2011).

1.3.2. Criterios SMART del objetivo general

La formulación del objetivo se organiza utilizando los criterios SMART. Este enfoque fue propuesto por Doran como una forma de establecer objetivos específicos, medibles y acotados temporalmente (Doran, 1981). La dimensión de aprendizaje asociada a los verbos utilizados se complementa con la Taxonomía de Bloom revisada por Anderson y Krathwohl (Anderson & Krathwohl, 2001).

- **Específico (Specific):** El alcance se limita a dos distribuciones ligeras de Kubernetes, K3s y MicroK8s, implementadas en configuraciones de Alta Disponibilidad de tres nodos cada una. El estudio se concentra en la energía contabilizada a nivel del host, el rendimiento y la resiliencia, sin pretender caracterizar todo el ecosistema Kubernetes.
- **Medible (Measurable):** El comportamiento de ambas plataformas será caracterizado mediante métricas cuantitativas de energía contabilizada y potencia media del host, utilización de CPU y memoria, *throughput*, latencia, tasa de error, tiempo de detección, tiempo de recuperación y solicitudes afectadas ante fallos controlados. La energía será obtenida mediante Scaphandre, las métricas de recursos mediante Node Exporter y las métricas de carga y respuesta mediante *k6*. La eficiencia energética se analizará relacionando la energía contabilizada con una unidad de trabajo común, utilizando inicialmente la energía por solicitud válida.
- **Alcanzable (Achievable):** La experimentación se ejecutará sobre una infraestructura virtualizada controlada y homogénea, compuesta por máquinas virtuales administradas mediante QEMU/KVM. El aprovisionamiento de la infraestructura será automatizado mediante Terraform y la configuración de los sistemas y clústeres será gestionada mediante Ansible. Estas herramientas permiten reducir la dependencia de configuraciones manuales y facilitar la reconstrucción del entorno experimental (Ansible Community, 2026; HashiCorp, 2026).
- **Relevante (Relevant):** El estudio busca aportar evidencia cuantitativa para la selección de una distribución ligera cuando la eficiencia energética, el rendimiento y la resiliencia constituyen restricciones relevantes para el diseño y operación de infraestructuras de Kubernetes.
- **Temporal (Time-bound):** El objetivo general se desarrollará dentro del periodo establecido para el proyecto de título y se organizará en tres fases: (i) implementación y validación de la infraestructura y de ambos clústeres HA; (ii) diseño y ejecución del protocolo experimental; y (iii) análisis estadístico, contraste con evidencia secundaria y redacción de resultados. Los hitos y plazos específicos serán definidos y controlados mediante el backlog del proyecto.

1.3.3. Objetivos Específicos

Los siguientes objetivos específicos descomponen progresivamente el objetivo general y se alinean con las etapas técnicas y metodológicas definidas para el proyecto:

1. **OE1 – Implementar** clústeres de Alta Disponibilidad de tres nodos utilizando K3s y MicroK8s sobre una infraestructura virtualizada homogénea mediante QEMU/KVM, automatizando su aprovisionamiento con Terraform y su configuración con Ansible, e incorporando un mecanismo de acceso redundante al plano de control mediante una IP Virtual.
2. **OE2 – Diseñar y ejecutar** un protocolo experimental reproducible que someta ambos clústeres a escenarios de carga equivalentes, registrando energía y potencia del host mediante Scaphandre, utilización de CPU y memoria mediante Node Exporter, métricas de rendimiento de la aplicación mediante *k6* y variables asociadas a la recuperación ante fallos mediante los mecanismos de observación y registro disponibles en Kubernetes.
3. **OE3 – Analizar estadísticamente** las diferencias observadas entre K3s y MicroK8s, calculando

medidas de tendencia y dispersión y seleccionando pruebas estadísticas coherentes con las características de los datos, con el propósito de determinar si las diferencias observadas son estadísticamente significativas.

4. **OE4 – Contrastar** los resultados experimentales obtenidos en la infraestructura local con evidencia disponible en la literatura sobre entornos de nube pública, identificando las condiciones bajo las cuales los resultados pueden utilizarse como referencia y explicitando las limitaciones de dicha comparación.

1.3.4. Matriz de trazabilidad

La matriz de trazabilidad vincula, una a una, las cuatro brechas numeradas en la Sección 1.2 (*Brechas identificadas*) con los objetivos específicos y sus respectivos indicadores de cumplimiento.

Brecha o problema (Sección 1.2)	Objetivo vinculado	Indicador de cumplimiento
Brecha 1: Existen estudios que comparan distribuciones ligeras de Kubernetes en consumo de recursos y rendimiento, pero no bajo una configuración de Alta Disponibilidad controlada específica para K3s y MicroK8s.	OE1 y OE2: Implementar ambos clústeres y ejecutar escenarios equivalentes.	Dos clústeres HA operativos, configurados con recursos equivalentes y sometidos al mismo protocolo experimental.
Brecha 2: La arquitectura HA introduce mecanismos adicionales de replicación, consenso y recuperación cuyo comportamiento energético y de resiliencia debe formar parte del escenario experimental.	OE1, OE2 y OE3: Implementar HA, medir su comportamiento y analizar estadísticamente las diferencias, incluyendo la recuperación ante fallos.	Energía contabilizada por ejecución, potencia media, energía por solicitud válida, tiempo de detección, tiempo de recuperación y solicitudes afectadas durante un fallo controlado de un nodo.
Brecha 3: La observabilidad energética a nivel de contenedor presenta limitaciones documentadas, por lo que se requiere definir explícitamente el nivel de medición y evitar atribuir directamente la energía estimada de cargas individuales al consumo energético de la plataforma física.	OE2: Realizar la medición energética a nivel de host, definiendo explícitamente el nivel de medición utilizado.	Medición energética realizada en el host mediante Scaphandre y los contadores energéticos RAPL/powercap y nivel de medición documentado.
Brecha 4: La evidencia experimental local y la evidencia reportada para nube pública deben mantenerse conceptualmente separadas, debido a que en IaaS el hardware físico subyacente no se encuentra bajo control del consumidor.	OE4: Contrastar los resultados con evidencia secundaria de IaaS.	Comparación contextual con fuentes identificadas y delimitación explícita de sus restricciones.

Cuadro 1.1: Matriz de trazabilidad entre las brechas identificadas (Sección 1.2) y los objetivos

1.4. Análisis de Requerimientos

El análisis de requerimientos transforma las necesidades identificadas en condiciones técnicas y metodológicas verificables. Se distinguen requerimientos funcionales, no funcionales y experimentales, de manera que las decisiones de implementación no se confundan con los requisitos que las justifiquen.

1.4.1. Requerimientos funcionales de arquitectura y orquestación

- **RF-01 – Implementación de K3s HA (OE1):** El sistema deberá permitir la implementación de un clúster K3s de tres nodos servidores utilizando *etcd* embebido. La documentación oficial establece que los clústeres HA con *etcd* embebido requieren tres o más nodos servidores y un número impar de servidores para mantener el quórum (K3s Project Authors, 2026).
- **RF-02 – Implementación de MicroK8s HA (OE1):** El sistema deberá permitir la implementación de un clúster MicroK8s de tres nodos con Alta Disponibilidad habilitada y Dqlite como almacén distribuido. Canonical establece que la HA se habilita automáticamente en clústeres de tres o más nodos (Canonical, 2026a).
- **RF-03 – Acceso redundante al plano de control (OE1):** El entorno deberá proporcionar una IP Virtual para representar el punto de acceso al plano de control. La selección de VRRP se fundamenta en que el protocolo permite transferir dinámicamente la responsabilidad de una dirección virtual hacia un nodo de respaldo cuando el nodo maestro deja de estar disponible (Nadas, 2010).
- **RF-04 – Despliegue de una carga de trabajo común (OE1, OE2):** Ambos clústeres deberán ejecutar la misma aplicación de referencia y exponerla mediante una interfaz equivalente para asegurar que la carga aplicada sea comparable.
- **RF-05 – Homogeneidad de virtualización (OE1):** Las máquinas virtuales utilizadas para K3s y MicroK8s deberán ejecutarse sobre el mismo entorno QEMU/KVM y utilizar configuraciones equivalentes de vCPU, memoria, almacenamiento y red. Las diferencias entre las plataformas deberán limitarse a los componentes propios de K3s y MicroK8s y a las configuraciones necesarias para su funcionamiento.

1.4.2. Requerimientos de automatización e infraestructura

- **RNF-01 – Reproducibilidad (OE1):** La infraestructura deberá poder ser reconstruida mediante código. Terraform permite definir recursos declarativamente y administrar su ciclo de vida mediante una configuración versionable y reproducible (HashiCorp, 2026).
- **RNF-02 – Estandarización de configuración (OE1):** La configuración de las máquinas virtuales y clústeres deberá gestionarse mediante *playbooks* de Ansible. La herramienta está orientada a automatizar configuración, despliegues repetibles y administración de sistemas (Ansible Community, 2026).

- **RNF-03 – Homogeneidad experimental (OE1, OE2):** Los nodos utilizados en las comparaciones deberán disponer de la misma asignación de vCPU, memoria, almacenamiento, sistema operativo, red y configuración base. Cualquier diferencia necesaria entre las distribuciones deberá corresponder únicamente a los componentes propios de cada plataforma.
- **RNF-04 – Versionado de la configuración (OE1):** Los archivos Terraform, Ansible, manifiestos de Kubernetes, scripts de carga y scripts de procesamiento deberán mantenerse versionados para permitir la reproducción de los experimentos.

1.4.3. Requerimientos de observabilidad energética

- **RE-01 – Medición energética del host (OE2):**

El entorno experimental deberá disponer de un mecanismo para registrar energía y potencia del host físico durante cada ventana experimental. La medición principal será realizada mediante Scaphandre, utilizando los contadores energéticos disponibles en la plataforma mediante RAPL/powercap (Hubblo, 2026a, 2026b).

- **RE-02 – Registro inicial y final (OE2):**

Cada ejecución experimental deberá registrar el valor inicial y final del contador energético utilizado, permitiendo calcular el incremento energético correspondiente a la ventana experimental.

- **RE-03 – Nivel de medición explícito (OE2):**

Deberá documentarse el dominio energético y nivel de observación representado por cada métrica utilizada. La energía contabilizada deberá interpretarse de acuerdo con el dominio energético representado por el contador utilizado y no deberá considerarse automáticamente equivalente al consumo eléctrico total de todos los componentes físicos del equipo (The Linux Kernel Documentation, 2022).

- **RE-04 – Validación de la medición (OE2):**

Antes de la ejecución definitiva de los experimentos se deberá verificar la consistencia entre la lectura directa del contador energético disponible en el sistema y la información registrada por Scaphandre durante una ventana temporal común.

1.4.4. Requerimientos de observabilidad de recursos y rendimiento

- **RO-01 – Medición de utilización de CPU (OE2):** El entorno deberá registrar la utilización de CPU de los nodos durante cada ejecución experimental. Para ello se utilizará Node Exporter, cuyos datos podrán ser recolectados y consultados mediante Prometheus (Prometheus Authors, 2026a, 2026b).
- **RO-02 – Medición de memoria (OE2):** El entorno deberá registrar la utilización de memoria de los nodos durante cada ejecución experimental. La medición deberá conservar marcas

temporales compatibles con las demás fuentes de observación, permitiendo relacionar el comportamiento de memoria con la carga aplicada y la energía contabilizada (Prometheus Authors, 2026b).

- **RO-03 – Almacenamiento de series temporales (OE2):** Las métricas de observabilidad deberán conservarse como series temporales mediante Prometheus, de forma que puedan consultarse y asociarse posteriormente con cada escenario y repetición experimental (Prometheus Authors, 2026a).
- **RO-04 – Medición de *throughput* (OE2):** Cada ejecución deberá registrar la cantidad de solicitudes procesadas por unidad de tiempo. La herramienta de generación de carga deberá conservar el número de solicitudes realizadas durante cada escenario.
- **RO-05 – Medición de latencia (OE2):** Cada ejecución deberá registrar la duración de las solicitudes realizadas a la aplicación de referencia. Además del promedio, se podrán utilizar medidas de percentiles, como p95 y p99, para caracterizar la distribución de los tiempos de respuesta. *k6* proporciona métricas de duración de solicitudes y permite obtener estadísticas agregadas y datos detallados de las pruebas (Grafana Labs, 2026a, 2026b).
- **RO-06 – Tasa de error (OE2):** Cada escenario deberá registrar la proporción de solicitudes que no fueron procesadas correctamente. *k6* proporciona una métrica específica para representar la tasa de solicitudes fallidas (Grafana Labs, 2026a).
- **RO-07 – Sincronización temporal (OE2, OE3):** Las diferentes fuentes de observación deberán utilizar marcas temporales compatibles, de manera que las mediciones de energía, utilización de recursos y rendimiento puedan asociarse con la misma ventana experimental.
- **RO-08 – Identificación de ejecución (OE2, OE3):** Cada conjunto de datos deberá asociarse explícitamente con la distribución evaluada, escenario de carga, número de repetición y ventana experimental correspondiente, evitando mezclar observaciones pertenecientes a ejecuciones diferentes.

1.4.5. Requerimientos de evaluación experimental

- **REX-01 – Escenarios de carga (OE2):** El protocolo deberá definir al menos tres niveles de carga: estado inactivo (*idle*), carga media y carga alta. La intensidad, duración y número de usuarios virtuales de cada escenario deberán quedar definidos antes de la ejecución.
- **REX-02 – Réplicas experimentales (OE2, OE3):** Cada combinación de distribución y escenario de carga deberá ejecutarse múltiples veces bajo las mismas condiciones de experimentación. Cada ejecución deberá conservar sus registros originales de energía, potencia, rendimiento y utilización de recursos, permitiendo estimar la variabilidad de los resultados.
- **REX-03 – Periodo de estabilización (OE2):** Antes de cada ejecución deberá existir un periodo definido de estabilización del sistema, verificando que todos los nodos se encuentren operativos y que la recolección de métricas se encuentre activa.

- **REX-04 – Calidad de datos (OE2, OE3):** Se deberán validar marcas temporales, valores ausentes, frecuencia de muestreo, interrupciones de la recolección y anomalías antes de ejecutar el análisis estadístico. Además, deberá verificarse la correspondencia temporal entre las métricas obtenidas mediante Scaphandre, Node Exporter, Prometheus y *k6*, de manera que los valores utilizados en el análisis correspondan a la misma ejecución experimental.
- **REX-05 – Unidad de trabajo (OE2, OE3):** Deberá definirse explícitamente la unidad respecto de la cual se calculará la eficiencia energética. Como indicador principal se propone la energía contabilizada por solicitud válida, utilizando el número de solicitudes obtenido durante la misma ventana experimental.

La energía contabilizada durante una ejecución se calculará como la diferencia entre el valor final y el valor inicial del contador energético correspondiente:

$$E_{\text{exp}} = E_{\text{final}} - E_{\text{inicial}}$$

donde E_{inicial} corresponde al valor del contador al inicio de la ventana experimental y E_{final} al valor registrado al término de dicha ventana.

A partir de esta magnitud se calculará inicialmente la eficiencia energética mediante la energía contabilizada por solicitud válida:

$$EE = \frac{E_{\text{exp}}}{N_{\text{válidas}}}$$

donde $N_{\text{válidas}}$ corresponde al número de solicitudes procesadas correctamente durante la misma ventana experimental.

Adicionalmente, podrá calcularse la potencia media mediante:

$$P_{\text{media}} = \frac{E_{\text{exp}}}{\Delta t}$$

donde Δt corresponde a la duración de la ventana experimental.

La cantidad de solicitudes, la duración de las solicitudes y la tasa de error serán obtenidas a partir de los resultados de *k6*, procurando que sus marcas temporales coincidan con la ventana utilizada para el cálculo energético (Grafana Labs, 2026a, 2026b).

- **REX-06 – Análisis estadístico (OE3):** La prueba estadística definitiva deberá seleccionarse en función del diseño experimental y de los supuestos de los datos observados. No se asumirá de antemano una prueba específica, se documentará la prueba utilizada, su nivel de significancia y los supuestos evaluados.
- **REX-07 – Control de condiciones energéticas (OE2):** La configuración relacionada con la política de energía y comportamiento del procesador deberá mantenerse constante durante las

comparaciones. También deberán registrarse la versión del kernel, configuración de CPU, asignación de recursos virtuales y demás parámetros que puedan afectar el comportamiento energético.

1.4.6. Requerimientos de resiliencia

La resiliencia será evaluada mediante escenarios de fallo controlado sobre un único nodo del clúster. El protocolo evitará provocar simultáneamente una cantidad de fallos que implique la pérdida intencional del quórum requerido por la arquitectura.

- **RR-01 – Fallo controlado de un nodo (OE2):** El entorno deberá permitir provocar la indisponibilidad controlada de un nodo participante del clúster.
- **RR-02 – Registro temporal del fallo (OE2, OE3):** Se deberá registrar el momento de inicio del fallo y el momento en que el sistema detecta la condición de indisponibilidad.
- **RR-03 – Tiempo de recuperación (OE2, OE3):** Se deberá registrar el tiempo transcurrido entre el inicio del fallo y la recuperación del funcionamiento esperado del clúster y de la aplicación de referencia.
- **RR-04 – Impacto sobre la carga (OE2, OE3):** Cuando sea técnicamente posible, se deberá registrar la cantidad de solicitudes fallidas o afectadas durante el proceso de recuperación.

1.4.7. Requerimientos de contraste con nube pública

- **RC-01 – Evidencia secundaria (OE4):** El contraste con nube pública deberá utilizar únicamente fuentes identificables, preferentemente documentación técnica, artículos científicos o informes de proveedores y organismos reconocidos.
- **RC-02 – Comparabilidad explícita (OE4):** Cada dato externo deberá documentar su plataforma, tipo de instancia o entorno, métrica utilizada y condiciones de evaluación, evitando tratar como equivalentes mediciones obtenidas bajo metodologías diferentes.
- **RC-03 – Limitación metodológica (OE4):** Las conclusiones derivadas del contraste con nube pública deberán presentarse como evidencia contextual y no como una medición directa del consumo eléctrico de la infraestructura del proveedor, debido a la abstracción propia del modelo IaaS (Mell & Grance, 2011).

1.4.8. Evaluación crítica de alternativas

Las decisiones de diseño se justifican mediante comparación de alternativas y no únicamente por preferencia tecnológica.

- **Kubernetes estándar frente a distribuciones ligeras:** Las distribuciones ligeras se seleccionan porque el objetivo del proyecto es estudiar escenarios donde la sobrecarga de la plataforma constituye una variable relevante. La literatura experimental muestra diferencias entre las distribuciones en términos de utilización de recursos y rendimiento (Koziolek & Eskandani, 2023;

Yakubov & Hästbacka, 2025). Kubernetes estándar no será incluido en el experimento principal, ya que incorporarlo aumentaría el número de configuraciones estudiadas y ampliaría el alcance más allá de la comparación central entre K3s y MicroK8s. Su evaluación podrá considerarse como una extensión futura.

- **K3s frente a MicroK8s:** Ambas alternativas permiten construir HA con tres nodos, pero utilizan mecanismos diferentes para el almacenamiento del estado. K3s utiliza *etcd* embebido (K3s Project Authors, 2026), mientras que MicroK8s utiliza Dqlite en la configuración HA (Canonical, 2026a). Esta diferencia proporciona una dimensión arquitectónica concreta para el análisis.
- **Arquitectura de observabilidad:** La observabilidad experimental se implementará mediante herramientas especializadas para cada dimensión de medición. Scaphandre será utilizado para energía y potencia del host, mientras que Node Exporter permitirá obtener métricas de utilización de recursos y Prometheus proporcionará el almacenamiento y consulta de las series temporales (Hubblo, 2026a, 2026b; Prometheus Authors, 2026a, 2026b). Para el rendimiento de la aplicación se utilizará *k6*, debido a que incorpora métricas de solicitudes, duración y errores (Grafana Labs, 2026a). Esta separación reduce el riesgo de mezclar niveles de observación diferentes y permite relacionar temporalmente las distintas dimensiones del experimento.
- **Scaphandre frente a alternativas de observabilidad energética en Kubernetes:** Scaphandre se selecciona como herramienta principal debido a que el objetivo del estudio requiere obtener energía contabilizada a nivel del host físico mediante los contadores disponibles en la plataforma (Hubblo, 2026a, 2026b). Kepler se considera una alternativa relevante para observabilidad energética integrada con Kubernetes, pero su fortaleza está en proporcionar métricas en diferentes niveles de abstracción. Debido a que el presente estudio no busca realizar atribución energética a contenedores individuales, la medición a nivel de host resulta más coherente con el objetivo experimental (Pijnacker et al., 2025; Sustainable Computing IO, 2026).
- **Keepalived/VRRP frente a un balanceador dedicado:** La solución con IP Virtual evita introducir un conjunto adicional de nodos exclusivamente destinados al acceso al plano de control. VRRP está diseñado para proporcionar una dirección virtual cuya responsabilidad puede transferirse entre routers participantes ante una falla (Nadas, 2010). La elección final se deberá validar en el experimento considerando también su posible sobrecarga.
- **Terraform y Ansible frente a configuración manual:** La configuración manual se descarta como mecanismo principal de aprovisionamiento porque dificulta la repetición exacta del experimento. Terraform proporciona un enfoque declarativo y versionable para infraestructura, mientras que Ansible permite automatizar la configuración mediante *playbooks* (Ansible Community, 2026; HashiCorp, 2026).

Bibliografía

- Doran, G. T. (1981). There's a S.M.A.R.T. way to write management's goals and objectives. *Management Review*, 70(11), 35-36.
- Anderson, L. W., & Krathwohl, D. R. (2001). *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman.
- Nadas, S. (2010, marzo). *Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6* (RFC N.º 5798). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc5798.html>
- Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing* (inf. téc. N.º NIST SP 800-145). National Institute of Standards y Technology. <https://doi.org/10.6028/NIST.SP.800-145>
- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes: Lessons learned from three container-management systems over a decade. *ACM Queue*, 14(1), 70-93. <https://doi.org/10.1145/2898442.2898444>
- Casalicchio, E., & Iannucci, S. (2020). The state-of-the-art in container technologies: Application, orchestration and security. *Concurrency and Computation: Practice and Experience*, 32(17), e5668. <https://doi.org/10.1002/cpe.5668>
- The Linux Kernel Documentation. (2022). *Power Capping Framework*. Consultado el 3 de septiembre de 2026, desde <https://docs.kernel.org/power/powercap/powercap.html>
- Koziolek, H., & Eskandani, N. (2023). Lightweight Kubernetes Distributions: A Performance Comparison of MicroK8s, K3s, k0s, and MicroShift. *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE '23)*, 17-29. <https://doi.org/10.1145/3578244.3583737>
- Kubernetes Authors. (2024). *¿Qué es Kubernetes? | Conceptos de Kubernetes*. Consultado el 3 de septiembre de 2026, desde <https://kubernetes.io/es/docs/concepts/overview/what-is-kubernetes/>
- Pijnacker, B., Setz, B., & Andrikopoulos, V. (2025). Container-level Energy Observability in Kubernetes Clusters. *arXiv preprint arXiv:2504.10702*. <https://arxiv.org/abs/2504.10702>
- SFEIR Institute. (2025). *K3s vs K8s vs MicroK8s: Which Lightweight Distribution to Choose?* Consultado el 3 de septiembre de 2026, desde <https://institute.sfeir.com/en/articles/k3s-vs-k8s-vs-microk8s>
- Yakubov, D., & Hästbacka, D. (2025). Comparative Analysis of Lightweight Kubernetes Distributions for Edge Computing: Performance and Resource Efficiency. *Service-Oriented and Cloud Computing. ESOC 2025*, 15547, 81-95. https://doi.org/10.1007/978-3-031-84617-5_7
- Ansible Community. (2026). *Introduction to Ansible*. Consultado el 3 de septiembre de 2026, desde https://docs.ansible.com/projects/ansible/latest/getting_started/introduction.html

- Canonical. (2026a). *High Availability (HA) | MicroK8s Documentation*. Consultado el 3 de septiembre de 2026, desde <https://canonical.com/microk8s/docs/high-availability>
- Canonical. (2026b). *Recover from lost quorum | MicroK8s Documentation*. Consultado el 3 de septiembre de 2026, desde <https://canonical.com/microk8s/docs/restore-quorum>
- Cloud Native Computing Foundation. (2026). *Annual Cloud Native Survey 2025* (inf. téc.). CNCF. Consultado el 3 de septiembre de 2026, desde <https://www.cncf.io/reports/the-cncf-annual-cloud-native-survey/>
- Grafana Labs. (2026a). *Metrics | Grafana k6 Documentation*. Consultado el 4 de septiembre de 2026, desde <https://grafana.com/docs/k6/latest/using-k6/metrics/>
- Grafana Labs. (2026b). *Results output | Grafana k6 Documentation*. Consultado el 4 de septiembre de 2026, desde <https://grafana.com/docs/k6/latest/results-output/>
- HashiCorp. (2026). *What is Infrastructure as Code with Terraform?* Consultado el 3 de septiembre de 2026, desde <https://developer.hashicorp.com/terraform/tutorials/aws-get-started/infrastructure-as-code>
- Huang, K., Ali, D., & Sofia, R. C. (2026). *Greenness-Driven Scheduling in Far Edge Kubernetes: A CODE-CO Evaluation* (inf. téc.). fortiss GmbH.
- Hubblo. (2026a). *Explanations about host level power and energy metrics – Scaphandre*. Consultado el 3 de septiembre de 2026, desde https://hubblo-org.github.io/scaphandre-documentation/explanations/host_metrics.html
- Hubblo. (2026b). *PowercapRAPL Sensor – Scaphandre*. Consultado el 3 de septiembre de 2026, desde https://hubblo-org.github.io/scaphandre-documentation/references/sensor-powercap_rapl.html
- K3s Project Authors. (2026). *High Availability Embedded etcd | K3s Documentation*. Consultado el 3 de septiembre de 2026, desde <https://docs.k3s.io/datastore/ha-embedded>
- Prometheus Authors. (2026a). *Data model*. Consultado el 4 de septiembre de 2026, desde https://prometheus.io/docs/concepts/data_model/
- Prometheus Authors. (2026b). *Monitoring Linux host metrics with the Node Exporter*. Consultado el 4 de septiembre de 2026, desde <https://prometheus.io/docs/guides/node-exporter/>
- Sustainable Computing IO. (2026). *Kepler Metrics*. Consultado el 3 de septiembre de 2026, desde <https://github.com/sustainable-computing-io/kepler/blob/main/docs/user/metrics.md>

Apéndice A

Tablas

A.1. Matriz de objetivos específicos y requisitos definidos

Objetivo	Requisitos asociados	Cobertura
OE1	RF-01, RF-02, RF-03, RF-04, RF-05, RNF-01, RNF-02, RNF-03, RNF-04	Implementación de ambos clústeres HA, acceso redundante al plano de control y automatización reproducible de la infraestructura mediante Terraform y Ansible.
OE2	RF-04, RNF-03, RE-01, RE-02, RE-03, RE-04, REX-01, REX-02, REX-03, REX-04, REX-05, REX-07, RR-01, RR-02, RR-03, RR-04	Diseño y ejecución del protocolo experimental: medición energética del host, escenarios de carga, réplicas, calidad de datos y escenario de fallo controlado.
OE3	REX-02, REX-04, REX-05, REX-06, RR-02, RR-03, RR-04	Análisis estadístico de las diferencias observadas entre K3s y MicroK8s, incluyendo los indicadores de recuperación ante fallos.
OE4	RC-01, RC-02, RC-03	Contraste contextual de los resultados locales con evidencia secundaria de infraestructura IaaS en nube pública.

Cuadro A.1: Matriz entre los objetivos específicos y los requisitos definidos

A.2. Métricas e instrumentos de medición

Dimensión	Métrica	Unidad	Fuente / herramienta
Energía	Energía contabilizada	J	Scaphandre + RAPL/powercap
Energía	Potencia media	W	Scaphandre + RAPL/powercap
Energía	Energía por solicitud válida	J/solicitud	Cálculo a partir de Scaphandre y resultados de <i>k6</i>
Recursos	Utilización de CPU	%	Node Exporter + Prometheus
Recursos	Utilización de memoria	GB / %	Node Exporter + Prometheus
Rendimiento	<i>Throughput</i>	solicitudes/s	<i>k6</i>
Rendimiento	Latencia	ms	<i>k6</i>
Rendimiento	Tasa de error	%	<i>k6</i>
Resiliencia	Tiempo de detección del fallo	s	Eventos, registros y monitoreo del clúster
Resiliencia	Tiempo de recuperación	s	Eventos, registros y monitoreo del clúster
Resiliencia	Solicitudes afectadas	solicitudes	<i>k6</i> + registro temporal del fallo

Cuadro A.2: Métricas experimentales y herramientas de medición

Apéndice B

Enlace a backlog

<https://docs.google.com/spreadsheets/d/1BBvxy5O2bchPX32bLfcvAZIxatpPCwSWgb9XDpb6u4g/edit?usp=sharing>